

A* ALGORITHM

AIM:

To find the shortest path from a start node to a goal node in a graph using heuristics to guide the search efficiently.

```
1  import heapq
2  def a_star(graph, heuristic, start, goal):
3      queue = [(heuristic[start], 0, start, [start])]
4      visited = set()
5      while queue:
6          est_total, cost_so_far, current, path = heapq.heappop(queue)
7          if current == goal:
8              return path
9          if current in visited:
10             continue
11          visited.add(current)
12          for neighbor, weight in graph[current]:
13              if neighbor not in visited:
14                  new_cost = cost_so_far + weight
15                  est = new_cost + heuristic[neighbor]
16                  heapq.heappush(queue, (est, new_cost, neighbor, path + [neighbor]))
17      return None
18  graph = {}
19  heuristic = {}
20  nodes = int(input("Enter number of nodes: "))
21  for _ in range(nodes):
22      node = input("Node name: ")
23      edges = input(f"Enter edges from {node} (format: neighbor:cost space-separated): ").split()
24      graph[node] = [(e.split(':')[0], int(e.split(':')[1])) for e in edges]
25      heuristic[node] = int(input(f"Heuristic value for {node}: "))
26  start = input("Start node: ")
27  goal = input("Goal node: ")
28  path = a_star(graph, heuristic, start, goal)
29  print("Shortest path:", path)
```

RESULT:

The optimal path from the start to the goal node, if one exists, with minimized total cost based on actual and estimated distances.